

Examen Parcial

Nombres : Gerson Osvaldo
Apellidos : Segura Perez
Código : 22200043
Fecha : 20/06/2026

I. Analisis de Arquitectura y concurrencia

1. El patrón Repositorio como SSOT

sol:

El principio de única fuente de verdad establece que un dato debe tener un único origen y control en toda la aplicación. Aunque la base de datos local actúe físicamente como única fuente de verdad, el repositorio es quien gestiona y expone. El repositorio es una fachada en la arquitectura porque proporciona una interfaz unificada y simplifica un subsistema complejo. Para lograr que la UI no conozca el origen de los datos el repositorio implementa una estrategia de sincronización. Esta estrategia se basa en:

- Lectura inmediata de forma local desde el DAO del ROOM
- Petición en segundo plano en paralelo o inmediato después mediante una API externa
- Procesamiento y persistencia. Si la red falla, el flujo de ROOM sigue activo. Si la red es exitosa, el repositorio toma la respuesta e inserta en ROOM actualizando
- Actualización Automática: la UI se actualiza de forma automática

2. Corrutinas y Structured Concurrency

sol:

View Model Scope es un alcance de corutina predefinido en Android KTX. Ayuda a ~~gestionar~~ la gestión de memoria y las fugas en la programación ~~asíncrona~~ asíncrona. Si el usuario cierra la pantalla antes de la respuesta ocurre lo siguiente:

- Cancelación del Scope: se destruyen las actividades o fragmentos
- Propagación de la cancelación: la cancelación se propaga en las corrutinas hijas
- Aborto de petición en RED: la petición +HTTP se cancela
- Protección de fugas y errores

3. Flujos Reactivos (Cold vs Hot)

Sal:

Técnicamente un Flow convencional de flujo Frio significa que es un perezoso, no genera datos ni consume recursos hasta que un recolector se suscribe (collect), ejecuta su bloque de código desde cero para cada nuevo suscriptor y carece de memoria para almacenar el último estado emitido.

Por otro lado, un Stateflow es un flujo caliente que mantiene datos en memoria de forma independiente a la existencia de suscriptores y es capaz de compartir el mismo flujo de datos en distintos recolectores en paralelo.

Para el manejo de estados de la interfaz de Usuario en Android se prefiere categóricamente Stateflow debido a que está diseñado específicamente para representar estados persistentes y eficientes.

4. Inyección de Dependencias (DI) Manual

El mecanismo de inyección de ~~datos~~ Dependencia manual a través de la clase Application consiste en centralizar la creación y la gestión del ciclo de vida de los objetos globales de la aplicación como la base de datos Room, dentro de una clase personalizada que hereda de Application, la cual ejecuta una sola vez al arrancar la app y permanece viva en todo el ciclo de vida de la aplicación.

II. Sección de Completar

1. El flujo de datos que emite automáticamente cada vez que una tabla de Room cambia se implementa usando el tipo (Flow)
2. El operador (Dispatchers.Default) permite ejecutar tareas pesadas fuera del Main Thread, específicamente optimizado para tareas de CPU como el parseo masivo de datos
3. Para un servicio en primer plano de ubicación sea válido en Android 10+ debe aclararse, en el manifiesto con el tipo (Location)
4. la función (Application Context) de la clase Application es la que garantiza que componentes compartan la misma instancia del repositorio mediante el cast
5. El componente que permite transformar una API basada en callbacks antiguos en una función suspendible moderna se llama (Suspendable Cancellable Coroutine)
6. El archivo de metadatos ~~de~~ que describe la estructura de la app al sistema operativo se llama exactamente (AndroidManifest.xml)
7. En arquitectura AOSP, la capa que actúa como puente estándar entre el framework y los controladores se denomina (HAL)
8. El operador de flow que permite combinar múltiples fuentes para emitir un estado unificado es (combine)

IV Arquitectura del AOSP completa

1.

1. Aplicaciones / System Apps

2. Frameworks de la API de Java

3. Bibliotecas nativas de C/C++
y android Runtime-ART

4. Capa de Abstracción de Hardware - Hal

5. Nucleo Linux / Linux Kernel

2.

1. Aplicaciones: es la capa superior donde interactúa el usuario

2. Framework de la API: expone todo el conjunto de herramientas y APIs en lenguaje JAVA/ Kotlin que los desarrolladores utilizan

3. Bibliotecas Nativas de C/C++ y android Runtime-ART proporciona los servicios esenciales de bajo nivel de Sistema y entorno de ejecución

4. Capa de Abstracción de Hardware - Hal: define interfaces estandarizadas que exponen las capacidades del hardware del dispositivo

5. Nucleo Linux: Es la base de la arquitectura encargada de interactuar directamente con el hardware físico